

《编译原理》实验报告

姓名：李经雄

学院：计算机科学与技术学院

专业：计算机科学与技术

邮箱：3220105346@zju.edu.cn

报告日期：2024. 3. 14

Lab2 语义分析

一 实验内容

1.1 类型定义

在type.hpp和type.cpp中，定义了三种类型。

- 基本类型 PrimitiveType，额外定义了两个静态类成员Int和Void，分别表示最基本的类型方便后续不用再每次都创建新的指针。
- 数组类型 ArrayType，具有元素类型element_type和整形数组dims两个成员，分别表示元素是哪个基本类型和维度信息。本次实验中元素类型只可能是基本类型中的Int
- 函数类型 FuncType，具有return_type和param_types数组两个成员，返回类型一定是基本类型之一，参数类型则可以是数组类型或者整数类型。

因为所有的节点都以指针表达，同时还需要实现equals()函数，用于判断两个类型是否相等。留待实现的只有ArrayType的判断函数，先调用element_type的判断函数看元素类型是否相等，在判断维度信息是否一致。这里需要注意因为函数参数也可以是数组类型而且会省略第一维的参数，我用-1作为特殊标记，如果遇到维度大小等于-1则默认这个维度检查通过，这样就可以支持函数参数的类型检查了。

1.2 符号表实现

- 符号Symbol，每一个变量或者函数在声明的时候都会获得一个自己专属的Symbol指针，Symbol存放变量的名字、唯一名字、类型和作用域下标。
- 符号表SymbolTable，对于每个程序有且仅有一张符号表，最关键的成员变量是一个
`std::vector<std::unordered_map<std::string, SymbolPtr>> scope_stack;` 这个向量每个元素都是一张字典，类似栈一样实现作用域的功能。需要实现的是符号表的各种函数。
 - add_symbol：检查当前作用域是否有定义过重名变量，如果没有则新建一个Symbol并插入字典
 - find_symbol：仅在当前作用域或者全局查找Symbol，通过字典的find函数很容易实现
 - enter/exit_scope：仅仅是简单的压栈和弹栈操作，弹栈的话注意检查不是空栈。

1.3 类型检查

此部分是实验中最复杂的部分。首先在type_checker.hpp中新增AST节点定义的检查函数，同时我还增加一个类型指针用于存放当前函数返回类型。详细部分将在下一节详细介绍。

```
1 // to store current function return type
2 TypePtr current_func_return_type;
3 TypePtr checkIfStmt(AST::IfStmtPtr node);
4 TypePtr checkWhileStmt(AST::WhileStmtPtr node);
5 TypePtr checkExpStmt(AST::ExpStmtPtr node);
6 TypePtr checkPrimaryExp(AST::PrimaryExpPtr node);
7 TypePtr checkInitVal(AST::InitValPtr node, const TypePtr& target_type);
```

二 实现亮点

2.1 VarDef和InitVal类型检查实现

为了方便实现类型检查，定义两个辅助函数用于快速检查当前传入的类型是不是Int或者Void类型。

```
1 static inline bool is_int(const TypePtr &t) {
2     auto prim = std::dynamic_pointer_cast<PrimitiveType>(t);
3     return (prim && prim->basic_type == BasicType::Int);
4 }
5 static inline bool is_void(const TypePtr &t) {
6     auto prim = std::dynamic_pointer_cast<PrimitiveType>(t);
7     return (prim && prim->basic_type == BasicType::Void);
8 }
```

检查变量的初始化应该是最复杂的一个了，因为涉及到数组初始值的匹配。在checkVarDef中，根据node节点的信息创建对应的整数或者数组类型，再将创建好的symbol挂载到VarDef节点上。值得重点介绍的是checkInitVal部分。从checkVarDef函数传入待匹配的类型（主要是为了数组匹配），根据InitVal的种类分为两种不同情况。

- InitVal节点是Exp表达式，此时只能匹配int整数变量
- initVal节点是花括号列表，此时只能匹配array型变量

第二种情况比较复杂，于是创建一个新函数单独处理doCheckArrayInit，重点介绍这个函数。将花括号列表内的情形分为两种情况，第一种是Exp表达式，另一种是嵌套花括号，这里直接递归调用这个函数即可。遍历每一个花括号内每一个孩子，如果是标量Exp，则成功匹配数+1，这里要注意表达式类型一定要是int并且成功匹配数不能超过数组总元素数。如果是嵌套花括号，通过匹配最大子数组的方式，先构建出当前可匹配的最大子数组，则成功匹配数+最大子数组元素数（隐式补足0），然后递归调用这个函数即可，这里要注意只有当是多维数组的时候才可以嵌套匹配。

```
1 void TypeChecker::doCheckArrayInit(const std::vector<AST::InitValPtr> & sublist, const
  std::shared_ptr<ArrayType> & arrType, int lineno) {
2     int totalCount = 1;
3     for (auto d : arrType->dims) totalCount *= d;
4     int filledCount = 0;
5     for (auto child : sublist) {
6         auto initv = std::dynamic_pointer_cast<AST::InitVal>(child);
7         if (initv->kind == AST::InitVal::Kind::EXP) {
8             TypePtr t = check(initv->expr);
9             if (! is_int(t)) {
10                 std::cerr << "Error: array init scalar not int at line " << lineno <<
std::endl; exit(1);
11             }
12             filledCount++;
13             if (filledCount > totalCount) {
14                 std::cerr << "Error: too many init at line " << lineno << std::endl; exit(1);
15             }
16         } else {
17             if (arrType->dims.size() < 2) {
18                 std::cerr << "Error: nested { } but no deeper array dimension left" <<
std::endl; exit(1);
19             }
20             // 首先应该用已填充元素数去整除 dims 的维度 找到最大的可初始化数组大小
21             // 例如: int a[2][3][4] = { { {1,2,3,4}, {5,6,7,8} } }
22             // 假如已经处理完 {1,2,3,4} 下一个到 {5,6,7,8} 那么 filledCount = 4
23             // 现在用 filledCount = 4 去整除 dims[2] = 4
```

```

24 // filledCount % dims[2] == 0 but filledCount % (dims[1] * dims[2]) != 0
25 // 所以我们要新建一个数组类型 subArr[4] 去处理 {5,6,7,8}
26 int i = 0; int temp = 1;
27 for (i = arrType->dims.size()-1; i >= 1; i--) {
28     temp *= arrType->dims[i];
29     if (filledCount % temp != 0) {
30         break;
31     }
32 }
33 i++;
34 auto subArr = std::make_shared<ArrayType>();
35 subArr->element_type = arrType->element_type;
36 for (std::vector<int>::size_type j = i; j < arrType->dims.size(); j++) {
37     subArr->dims.push_back(arrType->dims[j]);
38 }
39 int subTotal = 1;
40 for (auto d : subArr->dims) subTotal *= d;
41 auto & subSubs = initv->subInitvals;
42 doCheckArrayInit(subSubs, subArr, lineno);
43 filledCount += subTotal;
44 if (filledCount > totalCount) {
45     std::cerr << "Error: too many init in bigger dimension, line " << lineno <<
std::endl; exit(1);
46 }
47 }
48 }
49 return;
50 }

```

2.2 LVal类型检查实现

左值表达式类型检查首先要判断变量是否被定义过，并且将symbol挂载到LVal节点上，后续处理分为两种情况。

- 真正意义上的左值，必须求值到整形变量
- 作为函数的实际参数，可以是整形变量或者是数组类型

实际处理中，我是通过查到LVal节点的dims成员变量判断是否为数组，如果是整数则直接返回int即可，如果是数组则检查维度大小信息是否越界，最后比较node和symbol的dims大小即可分辨LVal的类型究竟是整数还是数组类型，如果是数组类型，则需要新建一个ArrayType并返回这个新的指针供checkFuncCall函数内部使用。下面代码片段展示了构建新数组类型的过程。

```

1 size_t remain = arrT->dims.size() - node->dims.size();
2 if (remain == 0) {
3     return PrimitiveType::Int;
4 } else {
5     auto newArr = std::make_shared<ArrayType>();
6     newArr->element_type = arrT->element_type;
7     newArr->dims.assign(arrT->dims.begin() + node->dims.size(), arrT->dims.end());
8     return newArr;}

```

三 AI工具使用

- ChatGPT：将框架代码交给ChatGPT分析，并让他解释代码框架的思路和我所需要完成的任务。描述待完成工作并让ChatGPT生成可以采取的实现形式供参考
- Copilot：辅助完成代码实现和注释

四 建议

实验框架注释很详细很清晰，尤其是数组初始化列表那块的讲解真的很清楚，嵌套花括号初始化做之前感觉很复杂的内容没想到实际代码并没有太复杂。赞！